

AlphaAudit

A Benchmark for Auditing AI-Generated Trading Strategies

with a reference implementation and negative result on Indian equities

Ashwin Jain

Department of Computer Science and Engineering, Motilal Nehru National Institute of Technology
Allahabad, Prayagraj, India, ashwinaadi2007@gmail.com

1 August 2026

The paper in eight numbers

1,550	candidates generated by one local 7B model	15.6%	of profitable strategies reverse sign once costs are charged
40.7%	survive contact with real data	11	are bankrupted outright by costs
14.5%	both execute <i>and</i> trade	0 / 631	clear deflation at $N = 1,887$ trials
28 / 222	leak flags land on code that actually trades	0 / 7	auditor configurations beat random rejection

Code, fixtures, corpus and evaluation protocol:

github.com/Ashwin-aadi/Trivijaya-Quant-Research

Abstract

A language model that writes trading strategies is a machine for manufacturing plausible backtests, and the two failure modes that make those backtests worthless—lookahead bias and multiple testing—are precisely the two a naive evaluation cannot see. **We release AlphaAudit: a benchmark for measuring whether an auditor can tell, in advance and out of sample, which machine-generated strategies are worth acting on.** It comprises a survivorship-free Indian equity environment, a verified transaction-cost model, leaky and clean fixture suites, an audited 1,550-candidate corpus, a frozen one-shot evaluation protocol, and the reference implementation reported below.

Setup. A local 7B model (qwen2.5:7b-instruct-q4_K_M, one fixed prompt, temperature 0.8) generated $N = 1,550$ candidate strategies over a survivorship-free, point-in-time universe of the 100 most liquid NSE equities, rebuilt quarterly from exchange bhavcopy archives using only prior sessions. Each candidate was executed by a backtester that raises rather than warns on a stale signal, and priced by a cost model built for the Indian statutory regime—STT, exchange and IPFT charges, SEBI turnover fee, stamp duty, GST, and a per-scrip depository charge—applied at the rates in force on each trade’s own date. Three independently ablatable auditors judge each candidate: an AST-based leakage detector, a

statistical layer computing the Deflated Sharpe Ratio against a hash-chained trial ledger the generator cannot write to, and a local-LLM semantic layer reading each stated rationale against its own implementation ($\kappa = 0.589$ against 50 human labels).

The dominant failure mode of an LLM strategy researcher is not that it cheats—it is that it does not run. All 1,550 draws returned parseable, interface-conforming code; 631 (40.7%) survived contact with real data; 406 of those (64.3%) never took a position. **14.5% of the corpus produced a strategy that both executes and trades.** The auditor is rarely the binding constraint.

Costs. Net of costs, the best of 225 rankable strategies falls from a gross Sharpe of 1.3049 to 1.2807—and survives only because it barely trades. The median strategy loses 0.194 of Sharpe and 4.4% of annual return; the mean loses 0.587 and 58.1%, the gap being a tail churning above $80\times$ annual turnover. **35 of 225 (15.6%) reverse the sign of their Sharpe, and 11 are bankrupted outright.** None reverse the other way, as none can. Two of eleven standard academic factors also flip, and momentum, at a net Sharpe of 0.966, no longer beats equal-weighting the same universe. At an honest trial count of $N = 1,887$, **zero of 631 candidates attain a deflated Sharpe probability above 0.95;** the corpus maximum is 4.4×10^{-8} .

The null. Sweeping coverage from 1.0 to 0.05 over all seven layer combinations, scored on a holdout evaluated three times under logged authorisation and now permanently closed, **no configuration beats random rejection at matched coverage** (AUAP -1.20 to -1.43 against a random 95% interval of $[-1.22, -0.86]$; six fall below it). The one configuration that beat random in sample was the one that provably cannot—its ranking signal is a monotone function of the quantity it was scored against—and out of sample it fails with the rest. We report the null rather than tune past it, and note that it survived two subsequent corrections of our own pipeline—the entire cost implementation, and a sign error in the ranking itself: *the magnitudes moved, the conclusion did not*. It remains confounded with corpus degeneracy and cannot separate “*the auditor is uninformative*” from “*this corpus is too weak to test it*”.

Disposition. We report the reference implementation’s failure as the benchmark’s first entry rather than as a result to be improved upon. The environment, protocol and fixtures are independent of whether our auditor worked; a stronger generator can be scored against exactly the same apparatus and contest the second claim directly.

Keywords: backtest overfitting • deflated Sharpe ratio • lookahead bias • selective prediction • LLM agents • transaction costs • Indian equities

1 Introduction

Ask a large language model for a systematic equity strategy and it will give you one. It will give you three hundred. Each will arrive with a plausible economic rationale, clean code, and—if you backtest it naively—a Sharpe ratio that would embarrass most professional managers. This is not evidence of skill. It is evidence that the search space is large and that a backtest is a very weak instrument.

Two mechanisms account for nearly all of it. The first is *leakage*: the code sees information it could not have had. A signal computed from today’s close that trades at today’s close is not a strategy, it is a memory. The second is *multiple testing*: if N independent strategies are tried and the best is selected, the maximum of N draws from a mean-zero distribution is positive by

construction, and grows with $\log N$ (Harvey et al., 2016; Bailey and López de Prado, 2014). Neither mechanism announces itself. Both produce equity curves that look exactly like skill.

An LLM strategy generator makes both worse, and it does so in a specific and under-appreciated way. It does not merely raise N ; it raises N silently. A researcher who tries twenty ideas by hand remembers trying twenty. A generator that emits a candidate every few seconds, unattended, destroys the researcher’s ability to know their own denominator—and the Deflated Sharpe Ratio is meaningless without an honest one.

This paper builds the instrument, not the strategy. We construct an auditor A that maps a candidate strategy to a rejection decision, a reason, and a deflated performance estimate, with three independently ablatale layers: an AST-based static analyser, a statistical deflation layer backed by a tamper-evident trial ledger, and a local-LLM semantic layer that reads each strategy’s stated rationale against its own implementation. We then ask whether the auditor is worth having, by measuring *selective deployment*: if you act only on the fraction c of candidates the auditor most trusts, does realised out-of-sample performance improve as c falls? The area under that curve, AUAP, is our primary metric, and the null hypothesis—that A is no better than rejecting at random—is something we test rather than assume away.

We run the whole apparatus on Indian equities, a market chosen for a reason that turns out to matter more than expected. Indian statutory transaction costs are heavy, asymmetric between buy and sell legs, and include a per-scrip depository charge that does not scale with trade size. A cost model that treats them as a flat basis-point haircut is not a simplification; it is a different experiment.

What this paper is primarily offering. The auditor we build fails its own headline test. We nonetheless regard the paper’s central artifact as the *benchmark*, not the auditor, and we have written it that way. The reason is structural: an auditor is a method and methods are superseded, but the environment a method must be tested in—a survivorship-free universe, an honest cost model, a fixed holdout, known-leaky positive controls, and a protocol that prevents the test set from being optimised against—is infrastructure, and in this area it does not currently exist openly for any market, let alone for India. Our null is the benchmark’s first entry. It is a low bar, and it is deliberately published as one.

Contributions

1. **AlphaAudit, a frozen benchmark for strategy auditing:** environment, fixtures, audited corpus, one-shot evaluation protocol, and the AUAP metric with its random-rejection baseline (§10).
2. **A negative result about LLM strategy generation that is not about leakage.** Of 1,550 candidates, all 1,550 parsed and conformed to the engine interface; 631 executed against real data; 406 of those never traded. The auditor is not the binding constraint—the interpreter is (§6.1).
3. **A time-varying Indian transaction cost model,** verified line by line against a discount broker’s public calculator, wired *inside* the backtest engine and charged per leg. It reverses the sign of 15.6% of profitable candidates and bankrupts eleven (§6.3).
4. **An honest trial counter,** hash-chained and append-only, outside the generator’s write path, and the demonstration that at $N = 1,887$ nothing in the corpus survives deflation (§6.4).
5. **A pre-registered null, published as the benchmark’s reference entry.** The auditor does not beat random rejection at matched coverage, in any of seven layer configurations, on a holdout evaluated under logged authorisation and now permanently closed. We

report it, and we report that it survived two full corrections of our own pipeline—the cost implementation, and a sign error in the ranking (§6.6, §8).

2 Related work

Backtest overfitting. That the reported Sharpe of a selected strategy is biased upward is old news; quantifying the bias is not. [Bailey and López de Prado \(2014\)](#) derive the Deflated Sharpe Ratio, which adjusts an observed Sharpe for trial count, sample length, skewness and kurtosis by raising the threshold a strategy must clear rather than lowering its estimate. [Bailey et al. \(2017\)](#) introduce the Probability of Backtest Overfitting via combinatorially symmetric cross-validation. [Harvey et al. \(2016\)](#) argue that the conventional $t > 2$ hurdle should be roughly $t > 3$ once the published factor zoo is treated as one enormous multiple-testing exercise. [López de Prado \(2018\)](#) supplies the machinery we use for purged k -fold cross-validation with embargo. Our contribution is not to these statistics but to their *plumbing*: all of them require N , and none of the papers that define them address what happens when the researcher is a program that cannot remember how many times it has tried.

Transaction costs and the anomaly literature. [Novy-Marx and Velikov \(2016\)](#) show that a large share of published anomalies do not survive realistic US trading costs, and that the survivors are disproportionately the low-turnover ones. [Frazzini et al. \(2018\)](#) estimate real-world costs from live institutional trading and find them lower than the academic consensus for large-cap strategies. Both literatures are calibrated to US market microstructure. India’s cost structure differs qualitatively—Securities Transaction Tax on both legs, stamp duty on the buy alone, a flat depository fee on the sell—and to our knowledge no open Indian equivalent exists.

Selective prediction. The idea that a model should be allowed to abstain is well developed in classification ([El-Yaniv and Wiener, 2010](#); [Geifman and El-Yaniv, 2017](#)), where risk–coverage curves are standard. We import the framework and change what it measures: coverage is the fraction of *strategies deployed*, and risk is realised out-of-sample Sharpe. The translation is not free, and §5 discusses where it strains.

LLMs as code and research agents. Code-generating models are now reliable enough to produce syntactically valid, interface-conforming programs at scale ([Chen et al., 2021](#)), and finance-specific models exist ([Wu et al., 2023](#)). What is not established is whether the output constitutes research. Our corpus statistics (§6.1) are a direct measurement of that gap for one 7B open-weights model.

3 Infrastructure

Three components had to be built before any experiment could be run. Each is boring, and each is load-bearing in the sense that a defect in it would silently invalidate everything downstream. We describe them at the level of detail required to know whether to trust them.

3.1 A survivorship-free, point-in-time universe

The universe on date t must contain exactly the securities investable on date t , including those that later delisted. A universe built from today’s index membership and applied backwards embeds survivorship bias so thoroughly that no downstream correction can remove it.

Table 1. Data foundation. The universe is a liquidity proxy and is never described as the NIFTY 100.

Quantity	Value	Sample
Trading sessions in calendar	1,475	2019-01-01 to 2024-12-31
Bhavcopy sessions cached	1,359 / 1,359	zero gaps
Panel rows	2,287,802	2,697 symbols, EQ series
Quarterly rebalances	20	2020-01 to 2024-10
Universe size at each rebalance	exactly 100	20 / 20
Distinct names ever in universe	185	vs. 100 at any one time
Names in first universe, gone by last	41	the survivorship evidence
Development window	1,232 sessions	2020-01-01 to 2024-12-31
Holdout window	246 sessions	2025 calendar year

We did not use a published index membership list, because free point-in-time NIFTY 100 constituent history could not be sourced. Instead the universe is *rules-based*: at each quarterly rebalance, the 100 NSE equities with the highest trailing 126-session median traded value, computed using only sessions strictly before the rebalance date, with a buffer band to suppress rank churn. Nothing consults a membership list, so a company that collapses simply stops qualifying rather than vanishing from history.

The 41 names that were in the first universe and absent from the last are the direct evidence the construction is survivorship-free (Table 1). A biased universe would show that count near zero.

Prices come from NSE’s own end-of-day bhavcopy archives, adjusted for splits and bonuses. We cross-checked against `yfinance` and found a **17.47% disagreement rate on adjusted closes** across 35,757 aligned observations. Inspection of the worst offenders showed *exact* integer ratios—BAJFINANCE at precisely $10\times$, ASHOKLEY at precisely $2\times$ —for securities with no declared corporate action in the window. Genuine data noise is never exactly $10\times$. We designate the exchange file authoritative and report the discrepancy rather than reconciling it, because the reconciliation would be a fiction.

3.2 An engine that makes lookahead hard to express

Point-in-time discipline is enforced structurally rather than by convention. A strategy is never handed the price panel; it receives a `MarketView` already truncated to before its decision moment. Every signal carries an `information_available_at` stamp, and the engine raises `PointInTimeError`—an exception, not a warning—if that stamp is not strictly prior to the fill session. A signal formed at session t ’s close fills at session $t+1$ ’s *open*.

Returns are accrued by provenance: a position carried from the previous session earns the full close-to-close move including the overnight gap, while only the portion bought this morning earns from the open. Collapsing both into open-to-close would silently liquidate the book every evening, discarding most of the equity premium in the sample and making every long-horizon strategy look structurally unprofitable.

Deliberate self-sabotage. Three strategies that cheat on purpose are permanent fixtures: one uses next-day return directly, one fits a `StandardScaler` on the full sample before splitting, one selects its universe from end-of-period membership. Gross of costs they return Sharpe ratios of **12,098**, **2.33** and **1.23** respectively. Each holds its number when known data artifacts are removed, confirming it cheats for its intended reason rather than riding a data scar. Thirty honest strategies form the false-positive control. *We built the disease before building the test*

Table 2. Round trip on a Rs.1,00,000 delivery trade, verified against a discount broker’s public calculator. The calculator omits the depository charge, which is levied separately; the first total is therefore the directly comparable figure.

Component	Amount (Rs.)
Securities Transaction Tax, both legs	200.00
Exchange transaction charge + IPFT, both legs	6.14
SEBI turnover fee	0.20
Stamp duty (buy leg only)	15.00
GST at 18% on charges (not on taxes)	1.14
Total, broker-comparable	222.4812
Depository charge, retail mode (sell leg, per scrip)	15.34
Total, as charged in this study	237.8212

for it.

As a sanity check, equal-weighting the universe returns 19.98% annualised at 21.14% volatility over 1,232 sessions, with 7.64% annualised tracking error and 0.9334 correlation against the NIFTY 100 price index—close enough to be a market proxy, distant enough to reflect that it is an equal-weighted liquidity portfolio rather than a capitalisation-weighted index.

3.3 The Indian transaction cost model

Costs are charged *inside* the engine, per name and per side, at the statutory rates in force on each trade’s own date. The two legs are not symmetric—stamp duty falls on the buy alone, the depository charge on the sell alone—so a blended round-trip rate gets both wrong. Rates are stored as a schedule of complete epoch snapshots (three, reflecting the April 2024 NSE reduction and the October 2024 SEBI true-to-label change) rather than as deltas, so no edit can silently corrupt a neighbouring period.

Table 2 is pinned as a unit test, asserted to the paisa, so no future rate edit can move it silently. Slippage is modelled as linear in volume participation and impact as a square-root law (Almgren et al., 2005). **Both coefficients are engineering defaults, not calibrations to Indian data**; daily bars cannot support a calibration and we do not claim one. §6.3 reports what fraction of measured drag they are responsible for, which is the only honest way to bound the damage they can do.

The depository charge is not scale-invariant. At Rs. 15.34 per scrip per sell instruction, a hundred-name book rebalancing once costs Rs. 1,534 regardless of position size—15 basis points of a Rs. 10 lakh book, and 0.0015 basis points of a Rs. 100 crore one. Any conclusion about a diversified strategy is therefore partly a statement about assumed capital. We report this dependence explicitly rather than hiding it in a default.

4 The auditor

4.1 A_{static} : leakage by structure, not by syntax

The static layer parses candidate source with Python’s `ast` module and classifies findings by leak class, line number, and severity. Its two strongest rules are structural rather than syntactic: *did the constructor accept bulk data?* and *did the decision function read stored state instead of*

the point-in-time view? These mirror the engine’s own information boundary, which is why they catch cheats whose results look plausible; pattern-matching `shift(-1)` catches only the cheats that announce themselves.

Against the fixture suite the layer achieves recall 1.00 (3/3 leaky) and precision 1.00 (0/30 honest flagged). **This number should be discounted and we say so in the paper rather than the appendix.** With syntax-based detectors alone, recall was 1/3; the structural rules were added after observing that failure, and were then measured on the same three fixtures that motivated them. That is closer to training accuracy than to a test. The genuine out-of-sample evaluation is against generated code the detectors were never designed around (§6.2).

4.2 A_{stat} : deflation against an honest denominator

We compute the Deflated Sharpe Ratio of Bailey and López de Prado (2014), which converts an observed Sharpe $\widehat{SR}$ into the probability that it exceeds what N trials would produce by luck, given sample length T , skewness γ_3 and kurtosis γ_4 :

$$\text{DSR} = \Phi \left(\frac{(\widehat{SR} - SR_0)\sqrt{T-1}}{\sqrt{1 - \gamma_3\widehat{SR} + \frac{\gamma_4-1}{4}\widehat{SR}^2}} \right),$$

where SR_0 is the expected maximum Sharpe across N trials—a function of both N and the *dispersion* of the trials’ Sharpes. The mechanism deserves a plain statement, because it is routinely misread: deflation does not shrink the strategy’s Sharpe, it raises the bar. With widely dispersed trials, 200 attempts buy an expected best of 2.77 by luck alone, so an observed 2.0 is *worse* than chance would produce.

This is worthless without an honest N , and an LLM generator is a machine for losing count. We therefore maintain a **hash-chained, append-only trial ledger** outside the generator’s write path. Every draw increments it—including syntax errors, runtime failures, and retries. Verification walks the chain and raises on any broken link, naming the edited entry. Nothing under `src/generate/` opens the file.

One deliberate imperfection in the ledger

The ledger holds 1,877 entries but the true draw count is 1,887, because an early run recorded one entry per candidate rather than one per draw. We did **not** retroactively edit the chain to insert the missing ten. An append-only ledger that gets rewritten when found wrong is not tamper-evident. The correction is applied at the point of use, and the discrepancy is deliberate.

4.3 A_{sem} : does the story match the code?

The semantic layer presents `qwen2.5:7b-instruct-q4_K_M` with a candidate’s stated rationale and its implementation, and asks for one of four labels: `consistent`, `rationale_implementation_mismatch`, `unfalsifiable_mechanism`, or `unacknowledged_known_anomaly`. Validation is against 50 human-labelled items, labelled before the model was run.

Cohen’s $\kappa = 0.589$ (Cohen, 1960), raw agreement 38/50, at the top of the “moderate” band of Landis and Koch (1977). Per-class agreement is uneven: `consistent` 26/31, `unfalsifiable_mechanism` only 2/5. No prompt was tuned before this measurement and none was tuned after it. The disposition—keep the layer—was decided before the number was known.

One reconciliation is worth reporting because it is the ablation argument in miniature. The human reviewer labelled `leak_full_sample_scaler` as `consistent`, and that is *correct*: its rationale honestly describes what its code does, and its only defect is where the scaler was

fitted. It is caught by A_{static} and correctly passed by A_{sem} . **A leaky strategy can carry a perfectly honest rationale.** The layers are not redundant, and any evaluation collapsing them into one “flagged by something” signal would obscure that.

5 Experimental design

Generation. One model, one prompt (SHA256 digest `f307433c...`), temperature 0.8, candidate i seeded at $42 + i$. 1,550 draws in two batches, pooled only after asserting identical model tag, prompt digest, seed rule and window. Each candidate emits a rationale, a feature specification, entry and exit rules, and executable code conforming to the engine’s strategy interface. Every attempt increments the trial ledger.

Evaluation. Each candidate runs in a sandboxed worker with a 180-second bound over the development window, net of costs, at a Rs.10 lakh book.

The abstention curve. For coverage $c \in (0, 1]$, rank candidates by auditor confidence, retain the top c fraction, and let $P(c)$ be the mean realised out-of-sample Sharpe of the retained set. Then

$$\text{AUAP} = \int_0^1 P(c) \, dc,$$

evaluated on the grid $c \in \{0.05, 0.1, 0.2, \dots, 1.0\}$. The baseline is rejection at random at matched coverage, with a bootstrap 95% interval. **An auditor that is informative must produce $P(c)$ rising as $c \rightarrow 0$.**

Two design choices that shape the result, stated plainly. Flat candidates—those that executed but never traded—are excluded from the ranking population rather than ranked last. They are all tied at exactly zero and at 64% of the executed set would dominate every retained set. Ranking them last would have been equally defensible and would have produced a different, lower curve. Second, survivors handed forward to future work are defined by $A_{\text{static}} \wedge A_{\text{sem}}$, excluding the statistical layer, which rejects the entire corpus and would make the handoff empty by construction. **This is the one place we chose the definition that produces a non-empty answer, and we flag it as such.**

Holdout discipline. The 2025 calendar year was frozen before any holdout data was fetched, set to a calendar boundary rather than to the latest available session so that window length could not become a researcher degree of freedom. It has been evaluated three times, each under written authorisation logged with a timestamp: once on a gross-of-costs pipeline, once after that pipeline was found defective and rebuilt, and once after the ranking defect described in §6.6. The first evaluation is retired and its artifacts are frozen under a directory whose README forbids quoting them without the word “gross”; the second is superseded and retained beside the third.

Every one of the three was authorised in advance to repair a defect, and no parameter, threshold, prompt or model was changed after any of them. That is the property that matters, and it is a different property from the count. A holdout is contaminated by iterative optimisation against it, not by arithmetic being performed on it correctly more than once; but we recognise that “we re-ran it and the conclusion held” is exactly what a researcher would say in both the honest and the dishonest case, so we publish the full before/after of each correction and the diffs that produced them rather than asking to be believed. **The holdout is now permanently closed.**

Table 3. The funnel. $n = 1,550$ draws, one model, one prompt.

Stage	Count	Share
Draws requested	1,550	—
Returned parseable, interface-conforming code	1,550	100.0% of draws
Executed to completion against real data	631	40.7% of corpus
— of those, flat: ran 1,232 sessions, never traded	406	64.3% of executed
— of those, rankable: executed and traded	225	14.5% of corpus
Runtime error	894	57.7%
Timeout at 180s	25	1.6%

6 Results

6.1 The corpus: strategies fail as programs, not as research

Read the first row of Table 3 carefully. *Every* draw returned syntactically valid Python defining a conforming `Strategy` subclass with a rationale. The model is entirely reliable at producing code that looks right. The attrition happens on contact with data.

The 894 runtime errors are dominated by a single confusion: `AttributeError` (327) and `ColumnNotFoundError` (247) together account for 64% of failures, and both trace overwhelmingly to the model calling pandas methods on polars frames, or treating a wide frame as though it were long. The prompt states the distinction explicitly and carries a worked example. The model makes the error anyway, in roughly a fifth of all draws.

This relocates RQ1. The question “what fraction of LLM-generated strategies fail static leakage screening” presumes leakage is the binding constraint. On this corpus it is not. The auditor was never reached for 59% of the corpus, and for a further 26% it had nothing to deflate. **An auditor tuned to catch cheating is guarding a door most candidates never reach.**

6.2 Leak classes, and one number we do not believe

Static analysis needs no execution, so it ran over all 1,550 sources: 222 rejected (14.3%), of which 195 carried one class, 26 two, and one three. By class:

<code>snooped_parameter</code>	166	<code>boundary_crossing_window</code>	2
<code>future_indexing</code>	58	<code>survivorship_selection</code>	1
<code>target_in_features</code>	23	<code>full_sample_statistic</code>	0

We do not believe the zero. Full-sample statistics are the most common leak class in hand-written quantitative code, and a corpus of 1,550 candidates containing none of it is not plausible. The cause is almost certainly ours: to raise the executable rate from near zero, the prompt advises working in plain Python rather than in long polars expression chains, and `pl.col("x").mean()` over an unfiltered frame is precisely where this class arises. The class distribution above therefore describes *this prompt*, not LLM-generated strategies in general, and the zero should be read as evidence that the instrument is blind here—not as evidence the defect is absent.

Most leak flags land on code that never produced a result. Static analysis is a statement about source, not about realised PnL, and the two are correctly independent: a leak is a leak

Table 4. Gross to net over the development window. $n = 225$ traded candidates. Gross and net come from a *single run*—the engine records the pre-cost series alongside the post-cost one—so the drag is a measured quantity on one path.

Quantity	Value	Sample
Best Sharpe, gross	1.3049	225
Best Sharpe, net	1.2807	225
Mean Sharpe reduction	0.5867	225
Median Sharpe reduction	0.1943	225
Mean turnover, annualised	82.2×	225
Median turnover, annualised	17.6×	225
Mean cost drag, annualised	58.13%	225
Median cost drag, annualised	4.39%	225
Sharpe sign reversed, $+\rightarrow-$	35 (15.6%)	225
Sharpe sign reversed, $-\rightarrow+$	0	225
Ruined outright (equity reached zero)	11	631

whether or not the strategy went on to trade. But the *distribution* of where flags land is itself a measurement, and on this corpus it is stark.

Fate of the 222 static-rejected candidates	Count	Share
Runtime error — never ran	114	51.4%
Evaluated but flat — ran, never traded	79	35.6%
Timeout	1	0.5%
Evaluated and traded	28	12.6%

Only 28 leak findings in the entire corpus attach to a strategy that took a position.

A worked example makes the mechanism concrete. Candidate 053 is flagged `future_indexing` at confidence 1.0 for a `shift(-1)` whose result is then filtered onto a past session date—textbook future indexing, and correctly caught. But the block is unreachable: the guard preceding it requires 21 rows from a view that returns at most 20, so it is false on every session and the function exits before the leak executes. Had it been reached it would have raised twice over, on an invalid `date` construction and on a column absent from the pivoted frame.

The verdict and the flat return series are compatible, not contradictory. What the pairing shows is that **a leakage prevalence rate measured over generated source is not a deployment risk rate**. Reported as “14.3% of candidates leak”, the static layer sounds like it is intercepting a substantial hazard; reported as “28 of 1,550 candidates leak *and* trade”, the same measurement says the hazard is rare because the population is inert. Both numbers are true. Only the second is decision-relevant, and we are not aware of prior work that separates them.

The semantic layer labelled 90.1% of the corpus `consistent`, rejecting 154 (9.9%): 119 rationale–implementation mismatches, 21 unacknowledged known anomalies, 14 unfalsifiable mechanisms. It is not a detector that fires indiscriminately. Whether the 154 are the *right* ones is not established on this population, since κ was measured on a different one.

6.3 What Indian transaction costs did to the corpus

Mean and median diverge by a factor of thirteen on cost drag (Table 4). The mean is dragged by a tail of candidates churning above $80\times$ annual turnover; the median strategy loses 0.19 of Sharpe and 4.4% a year. Both are reported because quoting either alone misleads.

The best gross strategy is also the best net strategy—because it barely trades, at $2.8\times$ annual turnover and 0.4% annual cost. That is the finding in miniature: **cost survival selects for low turnover, not for skill**, which is the same conclusion [Novy-Marx and Velikov \(2016\)](#) reach for US anomalies, arrived at here through a very different door.

Standard factors are not spared. Running eleven textbook factors through the same pipeline as a positive control, drag rises monotonically with turnover in ten of eleven rows. Two factors flip from profitable to unprofitable. Momentum ([Jegadeesh and Titman, 1993](#)) survives at a net Sharpe of 0.9656—but equal-weighting the same universe returns 0.9615. **Net of Indian costs, the momentum premium on this universe is inside the noise.**

The eleventh row, and why the book size matters. The one factor that breaks monotonicity is inverse-volatility weighting: the *lowest* turnover of any active strategy (0.011 per session) and the second *highest* cost (10.18 bps per session). It holds a hundred names and nudges each daily, and the depository charge is levied per scrip regardless of size. Sweeping book size confirms the mechanism precisely:

Table 5. Depository charge contribution by book size, obtained by differencing against a no-DP ablation rather than recomputed from the rate.

Strategy	Book	Total bps/day	DP bps/day	DP share	Sharpe
inverse_volatility	Rs. 10 L	10.183	9.963	97.8%	−1.3792
inverse_volatility	Rs. 1 Cr	0.866	0.597	69.0%	+0.6359
inverse_volatility	Rs. 10 Cr	0.590	0.059	10.0%	+0.6922
momentum_skip_month	Rs. 10 L	1.685	0.066	3.9%	0.9656
momentum_skip_month	Rs. 10 Cr	3.944	0.001	0.0%	0.7506

The depository contribution falls $169\times$ across a $100\times$ book sweep, close to the inverse proportionality the model predicts, and **the strategy’s verdict reverses with assumed capital**: −1.38 at Rs. 10 lakh, +0.69 at Rs. 10 crore. Momentum runs the opposite way—its cost *rises* with book size as impact and slippage grow while the fixed fee shrinks. The two regimes cross over. Small books are killed by flat fees; large books by market impact.

We report the corpus at Rs. 10 lakh, which was the pre-existing default, and we did not change it after observing that it flips a sign. Changing a benchmark default in response to an inconvenient result is the pathology this paper exists to detect.

How much of the drag is our assumption? For a realistically-trading strategy, 79.6% of the drag is published statutory rate, 3.9% depository charge, and only 16.5% comes from the two coefficients we chose. **The headline is not hostage to our slippage and impact defaults.** For the near-zero-turnover market reference the proportions invert, but the absolute cost there is 0.1 bps and immaterial.

6.4 Deflation at an honest trial count

At $N = 1,887$, the deflated Sharpe probability exceeds 0.95 for **0 of 631** evaluated candidates. The corpus maximum is 4.4×10^{-8} . The best raw Sharpe in the corpus, 1.3049, carries a deflated probability of 5.9×10^{-5} .

Those two numbers should feel violently inconsistent, and the inconsistency is the point: apparent skill and statistical significance are different quantities, and the gap between them is exactly what a trial count of 1,887 buys.

Table 6. Holdout ablation, $n = 225$, ranked by AUAP. Random-rejection 95% interval $[-1.2208, -0.8600]$; population mean $P(1.0) = -1.0351$.

Layers	AUAP	$P(0.05)$	Beats random
semantic	-1.1967	-1.1347	no
static	-1.2529	-1.1176	no — <i>below the interval</i>
static + semantic	-1.2782	-1.1805	no — <i>below the interval</i>
statistical	-1.3497	-3.0643	no — <i>below the interval</i>
semantic + statistical	-1.3777	-3.2931	no — <i>below the interval</i>
static + semantic + statistical	-1.4184	-3.2931	no — <i>below the interval</i>
static + statistical	-1.4339	-3.2931	no — <i>below the interval</i>

Result: the statistical layer’s rejection rate is uninterpretable at this trial count. We state this as a finding rather than a caveat, because it is an empirical observation about the instrument and not a hedge about the data.

At $N = 1,887$ over 1,232 sessions, every evaluated strategy is rejected. Whether this reflects appropriate statistical correction or saturation of the test cannot be determined from these data. The deflation term grows with N while sample length is fixed, so at some ratio the test rejects everything regardless of merit. A 100% rejection rate is consistent with an honest correction *and* with a saturated one, and no measurement in this study separates them.

This bears directly on the ablation in §6.5, and more sharply than we first realised. The layer rejects all 631 strategies it scores, at a recorded confidence of exactly 1.0 in every case, so it carries *no* ordering information whatever among the candidates it scored: they all tie. Its only discrimination is between scored and unscored. The four configurations containing it are the four with the steepest fall at tight coverage. The rejection rate and the ranking failure are plausibly the same fact observed twice. Establishing which requires evaluating the layer across trial-count regimes, which this corpus cannot support because N is a property of the experiment rather than a free parameter.

The Probability of Backtest Overfitting could not be computed on the pooled corpus: the eleven ruined strategies produce short series and the common-length matrix loses too many columns. **We report it as unavailable rather than repairing it by dropping the ruined strategies**, which would be a post-hoc restriction not part of the intended protocol. On the twelve-fixture suite, where it can be computed, PBO is 0.588 (0.531–0.667 across split counts)—above 0.5 in every variant, meaning selecting the in-sample best yields a below-median strategy out of sample more often than not.

6.5 The abstention frontier

No configuration beats random rejection at matched coverage (Table 6, Figure 1). One sits inside the random interval; six sit below it, meaning they order strategies *worse* than chance. The null is not rejected.

Figure 1 makes a point the table cannot. The curves do not merely fail to rise as coverage tightens—all of them *fall*. Concentrating on the candidates the auditor trusts most selects a worse portfolio than taking everything. Whatever the layers are ranking on, it is not out-of-sample performance, and at high confidence it is mildly anti-correlated with it.

One diagnostic is worth the space. On development data, the **statistical** layer alone

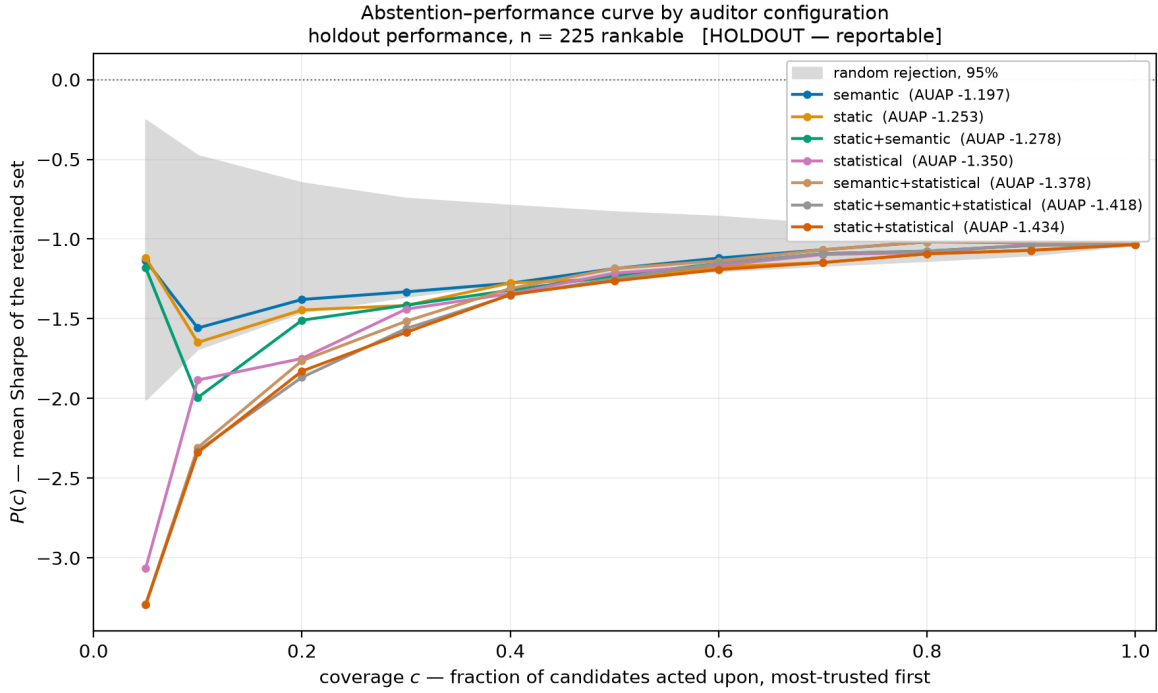


Figure 1. The abstention-performance curve, holdout, $n = 225$. An informative auditor produces curves that *rise* to the left: the more selective the system, the better what it keeps. Every configuration falls instead, and the four containing the statistical layer fall steeply—at 5% coverage they select an average holdout Sharpe near -3 against a population mean of -1.04 . The grey band is random rejection at matched coverage, and it widens to the left because 5% of 225 is eleven strategies.

was the only configuration to beat random—and it provably cannot be evidence, because its confidence is $1 - \text{DSR}$ and DSR is a monotone function of the very development Sharpe that $P(c)$ measures. Ranking by a quantity and then scoring on that same quantity beats random by construction. Out of sample the comparison becomes legitimate, and the layer fails with the rest, landing fourth of seven and below the random interval, with the worst $P(0.05)$ of any single layer. **The circularity diagnosis was made by argument in advance and confirmed by experiment afterwards.**

The corpus also collapses wholesale from development to holdout: mean Sharpe $+ \sim 0.40$ gross in sample against -1.0351 net out of sample, with 216/221 positive in sample and roughly half positive out of it. The best out-of-sample performer, at $+1.3618$, had a development Sharpe of -1.6696 . **The single best out-of-sample strategy is one any sane process would have discarded in sample.** That is not a finding about the strategy. It is multiple testing appearing inside our own results, which is the most direct evidence we can offer that the phenomenon we set out to measure is real.

6.6 A sign error in our own ranking, and what it cost

The numbers in Table 6 are the second version. The first inverted the semantic layer’s contribution, and we describe the defect in full because a paper about detecting false results that concealed one of its own would be self-refuting.

Each auditor layer records a **confidence** alongside its verdict. The code that turned those into a ranking negated the confidence, on the assumption that it meant *confidence that something is wrong*. For the static layer it does—the field holds a finding count. For the statistical layer it does vacuously, since that layer rejects every candidate it scores at confidence

1.0. For the semantic layer it does not: the language model reports confidence in *its own label*, around 0.95 for a confident **consistent** and 0.85 for a confident defect. Negating it ranked the strategies that layer had condemned above the ones it had cleared. The Spearman correlation between the two rankings is -0.972 , and the set retained at 5% coverage went from eleven of eleven semantically rejected to zero of eleven.

Four of the seven configurations were affected. The other three are provably untouched: under the corrected rule the static and statistical layers produce numerically identical scores on this corpus, and the recomputed AUAP for all three agrees to every printed digit. Nothing else was changed—not the corpus, the labels, the backtests, the cost model, the trial counter, or the holdout contents.

What the defect did not do. It did not change the conclusion, and it did not flatter us. Every affected AUAP moved *down*—the best configuration went from -1.1441 to -1.1967 —and the identity of the best configuration did not change. The inverted ranking had been the more favourable one.

A second error, found while checking the first. An earlier draft of this section reported that the correction raised the number of configurations ordering strategies worse than chance from two to six. It did not. Six of the seven AUAPs were already below the random interval before the correction; the sentence counting them in the previous version of this paper said “two”, and its table annotated **static** at -1.2529 as below the interval while leaving **statistical** at -1.3497 unmarked. That was a transcription error in the prose, not in the measurement, and it is corrected here rather than folded into the ranking correction it did not belong to. The count is six of seven in both versions.

What it did do, and why it belongs in the results. On development data the correction raises $P(0.05)$ for the semantic layer from -0.1088 to $+0.2910$, and for **static + semantic** from -0.0918 to $+0.3142$, against a population mean of -0.1896 . A correctly-oriented semantic layer visibly concentrates the better strategies at tight coverage *on the data it was developed against*. The identical ranking on the holdout gives -1.1347 and -1.1805 . The gap between an in-sample selectivity that looks real and an out-of-sample one that is absent is this paper’s subject, and it appeared here by accident, in our own pipeline, while we were measuring it in someone else’s.

Why no test caught it. Each layer’s tests check its labels, which were right. The abstention tests check that a curve is computed correctly from a ranking, which it was. The defect lived only in the join: a ranking computed correctly from a quantity that meant the opposite of what the calling code believed. It was found not by testing but by attempting to reuse the ranking for a different purpose in a subsequent project—which is a general lesson about where this class of defect hides, and a limitation of static and unit-level checking that our own static layer shares. A regression test now asserts the join directly.

7 Do these conclusions depend on who wrote the strategies?

Every number above was measured on a corpus written by one local seven-billion-parameter model. The first objection a referee raises is that this paper measures that model rather than machine-written strategies in general, and nothing in the preceding sections distinguishes the two.

Table 7. AlphaAudit measurements by generator. Each arm is 20 strategies; the local corpus is 1550 draws, of which 225 executed and traded.

Quantity	Local 7B	GPT	Claude	Gemini
Executed and took a position	14.5%	100%	100%	100%
Static findings	28 of 225	0	0	1
Semantic rejections	—	2	6	5
Deflation rejections	—	20 of 20	20 of 20	20 of 20
Mean Sharpe lost to costs	0.5867	0.5004	0.5795	0.8120
Profitable gross, unprofitable net	15.6%	6 of 20	1 of 20	4 of 20
Mean holdout Sharpe	-1.0351	-0.9491	-0.5781	-1.4768

The frozen task specification of Section 5 was therefore issued verbatim to 3 frontier models through their consumer chat interfaces — 4 independent requests of five strategies each — and all 60 were passed through the identical funnel with no auditor parameter altered and no line of measurement code added. The auditor has been read-only since the release of the companion stress benchmark, which is what makes this comparison possible at all. Design, hypotheses and predicted directions were pre-registered before any strategy was generated.

7.1 What changed, and what did not

Executability rises to saturation. Every one of the 60 frontier strategies parsed, executed over the full window and took a position, against 14.5 per cent of the local corpus. The dominant failure mode of Section 6.1 — strategies that fail as programs rather than as research — is a property of the small local model and disappears entirely.

The static auditor almost stops firing. It raised 1 finding across all 60 frontier strategies, against 28 of 225 rankable local candidates. We had pre-registered the opposite — that a more capable generator would leak just as often, in subtler ways — and that prediction is falsified. The single finding was a snooped parameter, not the class we predicted would dominate.

We cannot distinguish a robust detector from an inert one on this evidence. One finding in 60 is consistent with frontier models genuinely not leaking, and equally consistent with a detector whose patterns do not match how they write. Distinguishing the two needs deliberately leaky frontier-authored fixtures, which we do not have. Reported as an open question rather than as a result in either direction.

Costs bite regardless of author. The cost model removes 0.5004 to 0.8120 of Sharpe per arm, against 0.5867 locally, and turns 11 of 60 frontier strategies from profitable before costs into unprofitable after — a rate comparable to the 15.6 per cent measured on the local corpus. The best gross Sharpe on any arm, 1.0952, becomes 0.9315 net.

7.2 Deflation, and the holdout

Not one of the 60 frontier strategies clears a Deflated Sharpe of 0.95 at its own arm’s trial count, on development data or on the holdout. Neither does a single one of 1000 size-matched subsamples of the local corpus when scored on the holdout, against 41 in 1000 on development data. The bar is not merely uncleared by the frontier arms; out of sample it is uncleared by

everything.

Mean holdout Sharpe is -0.9491, -0.5781 and -1.4768 across the three arms, against -1.0351 for the 225 local rankable strategies. Every arm loses ground between the development window and the holdout, which is the ordinary out-of-sample decay this apparatus exists to measure.

7.3 The abstention frontier, as an exploratory analysis

The following is exploratory: AUAP appears nowhere in the generator-validation pre-registration, and was computed only after the fact, from holdout return series already spent. Coverage granularity is one strategy in twenty against one in 225 locally, so the curves are correspondingly coarse.

No auditor combination beats random rejection on any arm — 0 of 21 arm-by-combination cells. The best combination is semantic at -0.9717 on the GPT arm against a random interval of [-1.4806, -0.5319], -0.6083 on Claude against [-0.8711, -0.3004], and -1.7411 on Gemini against [-2.2476, -0.8580]. Locally the best was semantic at -1.1967 against [-1.2208, -0.8600]. *The central null of this paper replicates on all three frontier populations*, which is a stronger statement than the paper could make on its own corpus.

Two structures recur in every population, both exploratory. Adding the statistical layer to the ranking makes selectivity actively harmful: the single most-trusted strategy is worse than the average one, on the local corpus and on every arm. And the static layer contributes no ordering information at all — on one arm every combination containing it is identical, to four decimal places, to the same combination without it. A detector that flags almost nothing cannot reorder anything, which is the ablation restating the finding above.

7.4 What this does and does not license

The measurements that describe *the market* — what costs remove, what deflation rejects, what survives out of sample, whether abstention helps — are unchanged by a change of generator. The measurements that describe *the programs* — whether they run, whether they leak detectably — change substantially. Frontier models write better programs and, on this evidence, no better strategies.

The arms are identified by product name and effort setting as their interfaces reported them, not by version string. This study is reproducible as a procedure and not as a set of subjects: the same prompt issued in six months would reach different weights under the same names. That is a real limit of collecting frontier arms through consumer interfaces on a zero-rupee budget, and it is why the claims here are about a dated observation rather than about vendors.

8 Negative results

This section exists because the charter governing this work requires it, and because a paper that reports only what worked is the artifact this paper is about.

1. **AUAP does not beat random rejection**, in seven of seven configurations, out of sample, under a pre-registered protocol. Six configurations are worse than chance.
2. **The null survived two full corrections of our own pipeline.** The first evaluation was gross of costs because no cost model existed. We built one, wired it in, and re-ran everything downstream. The magnitudes moved substantially—holdout mean from -0.2755 to -1.0351 —and the conclusion did not move at all. The second correction is described in §6.6: a sign error that had inverted the semantic layer’s contribution to the ranking. Every affected number moved, and the conclusion again did not. We consider this the most

informative single fact in the paper, because it distinguishes “we found a null” from “we found a null and then twice checked whether it was an artifact of our own defect.”

3. **This null is confounded with corpus degeneracy**, and we cannot resolve the confound. The 225 rankable strategies are tightly clustered and mostly weak. A null therefore cannot distinguish “*the auditor is uninformative*” from “*this corpus is too weak to test the auditor*”. Both explanations survive the data. This caveat travels with the number permanently, and it is the reason the benchmark is released.
4. **The tracking-error decomposition failed.** Total tracking error against the index is 0.0775 over 1,231 sessions, but the two legs of the attempted decomposition came out at 0.1152 and 0.1463—each exceeding the total they were meant to partition. The traded-value proxy does not stand in for capitalisation weighting. Only the total is reported. Nothing was adjusted to make the numbers add up.
5. **The one suggestive signal we reported did not survive our own correction, and we record that it existed.** Before the ranking defect of §6.6 was found, the semantic layer alone reached $P(0.05) = -0.5147$ against a population mean of -1.0351 —its most confident 5% appearing to do roughly twice as well as average. We declined to build on it at the time, on the grounds that five per cent of 225 is eleven strategies and the random interval’s width said it was noise. It was worse than noise: it was the inverted ranking, and those eleven strategies were the ones the layer had *rejected*. Corrected, the same figure is -1.1347 , slightly *below* the population mean. **The suggestive signal is gone and nothing replaced it.** We report the episode rather than the corrected number alone, because the useful part is that a caveat stated in the right direction still understated the problem.
6. **27 of the 174 surviving strategies are not deterministic functions of their inputs.** Run one again, on the same panel with the same code, and its Sharpe changes—by up to 2.5352. Two mechanisms, both confirmed in source: unseeded randomness in 26 of the 27, and `set` iteration order, which Python randomises per process, in 27 of 27. The clearest instance keeps an arbitrary ten of its qualifying symbols via `list(set(...))[:10]`, giving Sharpe 0.7652, 0.4566 and 0.6825 under three hash seeds. All 27 are generator output; all eleven hand-written factor fixtures are deterministic, which locates the defect in what the model wrote rather than in the harness that ran it. Survivor membership is unaffected—the auditors read source, and the only performance gate is a flatness threshold seven orders of magnitude away—and the null is unaffected, because noise makes an ordering harder to detect and cannot manufacture one. The AUAP figures are computed from these Sharpe ratios and carry the noise, so their trailing digits are not exact. **We report this as a property of LLM-generated strategies that the literature does not discuss: a generated strategy may not define a portfolio as a function of data at all, and no auditor layer we built detects it, because it is invisible in a single execution.**

A process failure we are obliged to report

During the cost re-run, six candidates were silently reclassified as runtime errors by a type-inference defect: summing over an empty book returns integer zero, which caused a column to be inferred as integer and raise on the first genuine float. **The exit code was zero.** Among the six was the candidate holding the highest holdout Sharpe in the previous evaluation.

It was caught only by comparing funnel counts against the retired gross run—a check imposed by review, not by the code. **Without that comparison this paper would**

have been wrong and would have looked fine. We report it because a paper about detecting silent failure that concealed its own would be self-refuting.

A second one, found after the benchmark was frozen

PYTHONHASHSEED is set nowhere in our repository, so the entire corpus was backtested with hash randomisation active and a different seed in every worker process. Our seeding module seeds Python’s `random` and NumPy; neither reaches a strategy that constructs its own generator, and neither can affect hash order, which must be fixed before the interpreter starts. Every auditor layer had tests. **No test ran the same strategy twice and compared the results.**

The gap survived all of Phase 1 and was found only when the follow-on project needed a variance across resampled histories to mean something—at which point a strategy whose score already varies for no reason becomes arithmetically indistinguishable from the quantity being measured. The check that would have caught it is one line. It was the absence of the check, rather than any individual script, that failed, and the charter we wrote for ourselves forbids exactly this.

A third, and the one that bears on the null

Of 150 corpus strategies compared in follow-on work, **11 clusters contain strategies whose realised net return series are identical on every session** to within 10^{-12} : different code, different stated rationale, identical behaviour. 27 strategies sit in a cluster; the largest holds four. A further 20 pairs correlate above 0.9999.

One cluster pairs a generated candidate with the equal-weight baseline it was meant to beat. The model rediscovered equal-weighting and presented it as novel with its own economic rationale. A_{sem} is specified to flag exactly this—“unacknowledged known anomaly”—and did not. That is one checkable miss found incidentally, not a measured false-negative rate, and we do not present it as one.

The generator’s failure mode is therefore not only that its strategies are weak but that **it writes the same weak strategy repeatedly**, and no auditor layer can see this, because every layer reads one candidate at a time. This is direct evidence for the corpus-degeneracy confound we flag against our own null in §6.5: a corpus with 27 behavioural duplicates in 150 has fewer effective candidates than its count implies, so an abstention curve computed over it ranks fewer distinct objects than the coverage axis suggests. The null stands as reported. The confound is now measured rather than suspected.

9 Limitations

1. **One generator, one prompt, one temperature.** Every corpus statistic describes `qwen2.5:7b-instruct-q4_K_M` at temperature 0.8 under a single prompt whose digest is published. These are not claims about LLM strategy generation in general. A stronger generator would plausibly change the funnel and might change the null.
2. **The prompt steers implementation style**, which biases the leak-class distribution downward in at least one class (§6.2).
3. **The corpus is smaller than its count.** 27 of 150 compared strategies are exact behavioural duplicates of another, so the effective number of distinct candidates the abstention curve ranks is lower than the coverage axis implies, by an amount we have not

corrected for. No auditor layer detects this, because each reads one candidate in isolation. Cross-candidate redundancy detection is an obvious addition to the auditor and we did not build it.

4. **The static layer’s perfect fixture score is provisional**, for the reason given in §4: its decisive rules were written after seeing it fail.
5. **Slippage and impact coefficients are uncalibrated**. They account for roughly 16% of drag on realistically-trading strategies at a Rs.10 lakh book, but they dominate in the large-book regime, so Table 5’s right-hand rows are more assumption-laden than its left-hand ones.
6. **Three historical statutory rates could not be sourced** and are held constant across the window; each is documented with the maximum error it could introduce, jointly under one part in a thousand of a round trip. Stamp duty before July 2020 was state-dependent and the uniform rate overstates cost in the first six months.
7. **The universe is a liquidity proxy, not the NIFTY 100**, because free point-in-time index membership could not be obtained. It is survivorship-free by construction, which was the binding requirement, but it is not the index and is never described as such.
8. **The statistical layer was evaluated at a single trial count**. Future work should evaluate it across different trial-count regimes; the result at $N = 1,887$ is reported in §6.4 rather than here, because it is an observation and not a caveat.
9. **PBO is unavailable** on the pooled corpus, for the reason in §6.4.
10. **Flat candidates are excluded from the ranking population** rather than ranked last; the alternative is defensible and would lower the curve.
11. **The holdout is a single calendar year** (246 sessions) of one market. Out-of-sample evidence of this length supports a direction, not a magnitude.
12. **Reported performance for 27 survivors is one draw, not a fixed quantity**, for the reason in §8. We did not re-run under a pinned seed, and would not: pinning replaces one arbitrary draw with another rather than with a correct value, because the defect is in the strategies and not in our measurement of them. Any downstream use of the surviving set should exclude them—and must report that doing so is not performance-neutral, since the excluded 27 average -0.6722 against the retained 147’s -0.1320 .

10 The AlphaAudit benchmark

The protocol is deliberately restrictive: a fixed shared holdout, **one evaluation per submission**, no tuning on holdout results, and no revision of the benchmark in response to submissions. Independent one-shot evaluations do not contaminate a frozen test set; iterative optimisation against it does, and a benchmark that permits the latter measures nothing after its first few users. Every reported AUAP must carry its trial count N and its generator, because an AUAP without the trial count that produced its corpus is not interpretable.

Availability. Everything in Table 8 is at github.com/Ashwin-aadi/Trivijaya-Quant-Research, together with the source of this paper. Start at `benchmarks/alphaaudit/RESULTS.md`, which carries every number in this study with its sample size, including the ones we could not compute and the reason why.

Table 8. What is released. Each component is usable independently of the others and of whether our reference auditor worked.

Component	Contents
Environment	survivorship-free liquidity universe, 20 quarterly rebalances, 185 distinct constituents; point-in-time engine raising on stale signals
Cost model	Indian statutory schedule, three epochs, verified to the paisa; participation slippage and square-root impact, coefficients exposed
Positive controls	3 leaky fixtures with documented cheats (Sharpe 12,098 / 2.33 / 1.23)
Negative controls	30 honest fixtures, for false-positive rate
Corpus	1,550 candidates, per-layer verdicts, outcomes, and return series
Human labels	50 hand-labelled rationale-code pairs (κ reference standard)
Metric	AUAP over the coverage grid, with a bootstrap random-rejection baseline
Protocol	fixed shared holdout, one evaluation per submission

What a submission must beat. The reference entry is the seven-configuration table in §6.5: best AUAP -1.1967 , against a random-rejection interval of $[-1.2208, -0.8600]$, on a corpus generated by `qwen2.5:7b-instruct-q4_K_M` at $N = 1,887$. This is a low bar. We publish it as one.

The honest reason a benchmark is the right artifact here. Our null is confounded with corpus degeneracy and we cannot resolve the confound (§8). A single group with a single 7B generator cannot. What can resolve it is a second entry using a stronger generator on the identical environment: if a capable generator produces a corpus on which some auditor beats random, the degeneracy explanation is settled and our null is correctly read as a statement about weak corpora. The benchmark exists because that is the experiment we cannot run alone.

11 The companion benchmarks

This benchmark is one of three built on a shared point-in-time universe, a shared Indian transaction-cost model and a shared backtest engine, released from a single repository. A strategy can therefore carry an audit verdict, a fragility score and a deployment capacity measured on the same data under the same assumptions, which is what makes the cross-benchmark statements in this paper possible.

- **RegimeStress** — when a surviving strategy breaks, measured across counterfactual regime paths. It supplies the fragility scores and the duplicate criterion.
[papers/pdf/regimestress.pdf](#) · [benchmarks/regimestress](#)
- **FlowState** — how much capital a strategy can absorb before it can no longer be traded within its own participation limit. It supplies the deployment-capacity figures.
[papers/pdf/flowstate.pdf](#) · [benchmarks/flowstate](#)
- **Generator validation** — the study of Section 7: the identical frozen task specification issued to three frontier models and passed through all three benchmarks unchanged, with its pre-registration, its falsified hypotheses and its full results.
[benchmarks/generator_validation](#)

The three were designed to be read in order — is it real, when does it break, how much can it take — and each answers a question the previous one raises. Every artifact, correction log and

evaluation protocol is in the repository: github.com/Ashwin-aadi/Trivijaya-Quant-Research.

12 Conclusion

We set out to determine whether an adversarial auditor can tell real trading signals from artifacts in a corpus written by a language model. On the evidence assembled here, for this generator, it cannot—not better than chance.

Three things were learned along the way that we did not expect to be the interesting part. The dominant failure mode of an LLM strategy researcher is not dishonesty but inertness: six candidates in ten never survive contact with data, and two thirds of the rest never trade. Indian transaction costs, honestly modelled, reverse the sign of one profitable strategy in six and reduce the momentum premium on a liquid universe to the width of the noise. And an honest trial count of 1,887 removes every trace of statistical significance from a corpus whose best member has a superficially respectable Sharpe of 1.28.

The infrastructure functions as designed, and it behaved consistently under repeated evaluation: it reported no signal because none was detectable in this corpus, and it reported the same thing after we corrected our own most serious implementation error. That invariance is the closest thing to a control a null result admits.

What we are left holding is therefore not a method but a measuring apparatus, and that is what we release. A benchmark whose reference implementation fails its own headline test is an unusual object to publish, but it is the honest one: the protocol, the fixtures, the corpus and the cost model are all independent of whether our particular auditor worked, and they are what another group would otherwise have to rebuild before disagreeing with us. It is all at github.com/Ashwin-aadi/Trivijaya-Quant-Research.

References

- Almgren, R., Thum, C., Hauptmann, E., and Li, H. (2005). Direct estimation of equity market impact. *Risk*, 18(7), 58–62.
- Bailey, D. H., and López de Prado, M. (2014). The Deflated Sharpe Ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5), 94–107.
- Bailey, D. H., Borwein, J., López de Prado, M., and Zhu, Q. J. (2017). The Probability of Backtest Overfitting. *Journal of Computational Finance*, 20(4), 39–69.
- Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*.
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.
- El-Yaniv, R., and Wiener, Y. (2010). On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11, 1605–1641.
- Frazzini, A., Israel, R., and Moskowitz, T. J. (2018). Trading costs. *SSRN Working Paper 3229719*.
- Geifman, Y., and El-Yaniv, R. (2017). Selective classification for deep neural networks. *Advances in Neural Information Processing Systems*, 30.
- Harvey, C. R., Liu, Y., and Zhu, H. (2016). ... and the Cross-Section of Expected Returns. *The Review of Financial Studies*, 29(1), 5–68.
- Jegadeesh, N., and Titman, S. (1993). Returns to buying winners and selling losers: Implications for stock market efficiency. *The Journal of Finance*, 48(1), 65–91.

- Landis, J. R., and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159–174.
- López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.
- Novy-Marx, R., and Velikov, M. (2016). A taxonomy of anomalies and their trading costs. *The Review of Financial Studies*, 29(1), 104–147.
- Wu, S., Irsoy, O., Lu, S., et al. (2023). BloombergGPT: A Large Language Model for Finance. *arXiv:2303.17564*.